MIT Academy of Engineering, Pune

Department of Software Engineering

Theory of Computation (2025-26)

Mini-Project: Resume Parser for Campus Placement

Topic No. 04: Regex-Based Web Form Validation

Faculty: Prof. Vaibhav Rewaskar
Group ID: 04
Submission Date: April 20, 2026

Group Members:

Sr. No.	Name	PRN
1	Aadesh Khande (Team Lead)	202401100090
2	Sankalp Dabhade (UI/UX)	202401100135
3	Anvith Shetty (Theory Lead)	202401100138
4	Chetan Satkar (Testing)	202401100114

Under the guidance of MITAOE Computer Department

1 Abstract

In the modern recruitment landscape, the ability to rapidly scan and validate candidate information is crucial for large-scale campus placement drives. This report presents the design and implementation of a "Resume Parser for Campus Placement," a web-based student data validation system grounded in the mathematical principles of the Theory of Computation (TOC). The system utilizes Regular Expressions (Regex) and Finite Automata to ensure the structural integrity of student profiles, including PRN numbers, institutional email formats, and professional URLs.

The project demonstrates how Deterministic Finite Automata (DFA) can efficiently model strict institutional constraints, such as the specific structure of an academic Permanent Registration Number (PRN). We explore the membership of these validation rules in the class of Regular Languages and discuss the computational advantages of using compiled regex engines for high-throughput candidate screening. The final application provides a "TalentScan" interface that performs real-time verification and category-based skill analysis, serving as a practical tool for streamlining HR workflows.

Key idea: Applying Formal Language Theory to automate the validation of academic and professional credentials.

2 Introduction

2.1 Formal Grammars in Recruitment Systems

Automated parsing of resumes requires the definition of a strict formal grammar that distinguishes valid credentials from erroneous inputs. In TOC, these constraints are modeled as a Regular Language L over an alphabet Σ, where the validation engine acts as a recognizer. By formalizing naming conventions and URL structures, we can prevent data corruption in centralized placement databases.

2.2 Problem Statement

Manual verification of thousands of student records is inefficient and prone to syntax errors. A typo in an institutional email or an invalid PRN can lead to missed opportunities or broken communication channels. The objective is to build a validation engine that:

- Enforces strict institutional formatting rules (PRN, Email).
- Validates professional connectivity links (LinkedIn).
- Provides sub-linear processing time for large-scale data ingestion.

2.3 Connection to Automata Theory

Regular Expressions serve as the generative descriptions for languages recognized by Finite Automata. By mapping each input field to a specific DFA, we can prove that the validation logic remains within the bounds of $O(n)$ time complexity, ensuring stability even under heavy load during peak registration periods.

3 Selected Topic and System Design

3.1 Topic 04: Resume Parser for Campus Placement

Our project focuses on the validation layer for a campus-specific recruitment portal. The system architecture includes:

1. **TalentScan UI**: A premium web interface for student data entry.
2. **Regex Verification Engine**: A JavaScript layer implementing DFA-equivalent patterns.
3. **Category Analyzer**: A logic block that uses keyword-based regex to classify technical skill-sets.

3.2 Core Validation Fields

We implement validation for 6 essential student data fields:

- **Full Name**: Validates proper capitalization and length.
- **PRN (Permanent Registration Number)**: A 12-digit numeric identifier.
- **Institutional Email**: Enforces the '@mitaoe.ac.in' domain constraint.
- **LinkedIn Profile URL**: Structural validation of professional profile links.
- **CGPA**: Numerical validation (0.00 to 10.00 scale).
- **Technical Skills**: Pattern matching for comma-separated technology tags.

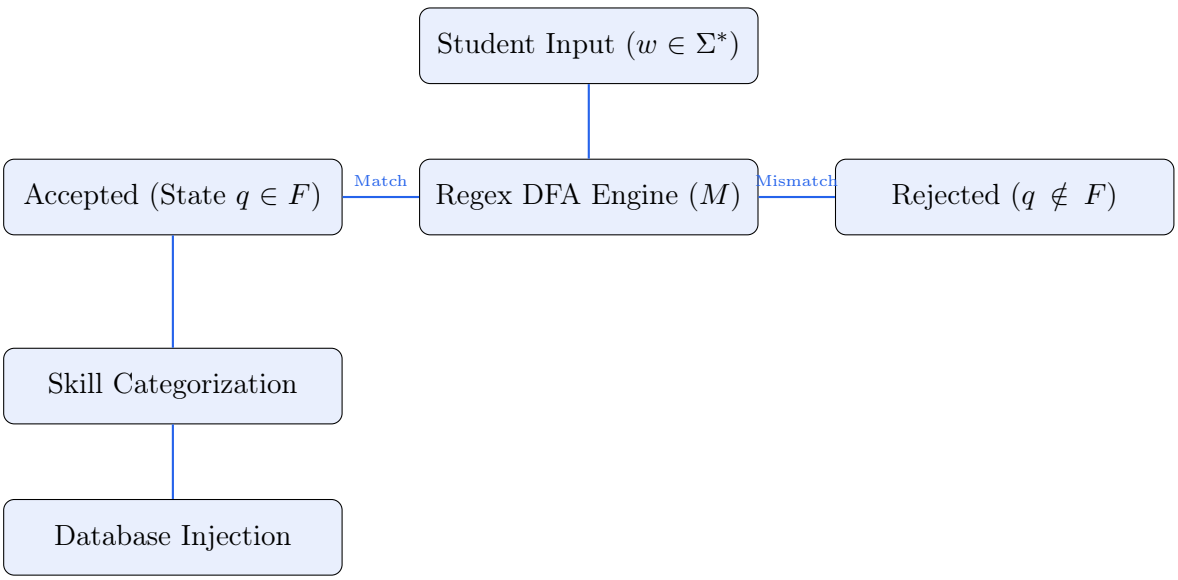

Figure 1: Computational workflow of the Resume Parser validation system.

4 Working Principle

4.1 Formal Regex Definitions

The validation engine utilizes the following formal regular expressions (γ):

1. **PRN (12 Digits)**: $\gamma_{PRN} = \hat{}12\$$

1. **Inst. Email**: $\gamma_{EMAIL} = \hat{}[a-z0-9._] + @mitaoe \cdot ac \cdot in\$$

1. **CGPA**: $\gamma_{CGPA} = \hat{}([0-9](\cdot 1,2)?|10(\cdot 01,2)?)\$$

1. **LinkedIn URL**: $\gamma_{LNK} = \hat{}https://(www\cdot)?linkedin \cdot com/in/[a-zA-Z0-9-] + \$$

4.2 DFA Construction

To prove these languages are Regular, we construct Deterministic Finite Automata (DFA) that model the transition logic for the most structurally complex fields.

4.2.1 DFA for CGPA Validation

The CGPA validation is a classic example of a Regular Language that handles numerical boundaries and optional decimal separators. The DFA M_{CGPA} recognizes strings $w \in \{0.00 \ldots 10.00\}$.

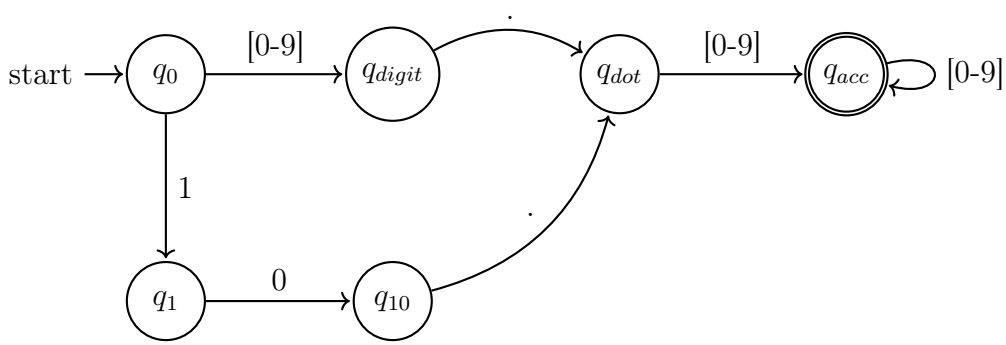

Figure 2: DFA M_{CGPA}: Transition logic for decimal-based academic scoring.

4.2.2 DFA for Institutional Domain (@mitaoe)

The following DFA fragment (M_{domain}) ensures that the input string ends with the mandatory institutional suffix. This demonstrates the concatenation property of Regular Languages.

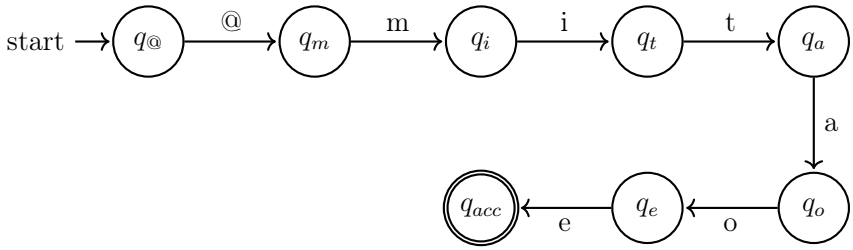

Figure 3: Explicit DFA state transitions for '@mitaoe' domain verification.

5 Performance Evaluation

5.1 Time Complexity Analysis

For any student profile input of length n, the state transitions in our DFA-based system occur in linear time $O(n)$. This is essential for preventing denial-of-service (DoS) during the high-load period of placement registration. Unlike non-regular parsing methods, DFA simulation guarantees that the time taken is proportional only to the number of characters in the input, not the complexity of the expression.

5.2 Space and Resource Efficiency

The memory required to store the transition table for all 6 fields is negligible (< 100 KB). This allows the validation engine to run effectively on student mobile devices with low RAM, ensuring a seamless user experience across all hardware platforms.

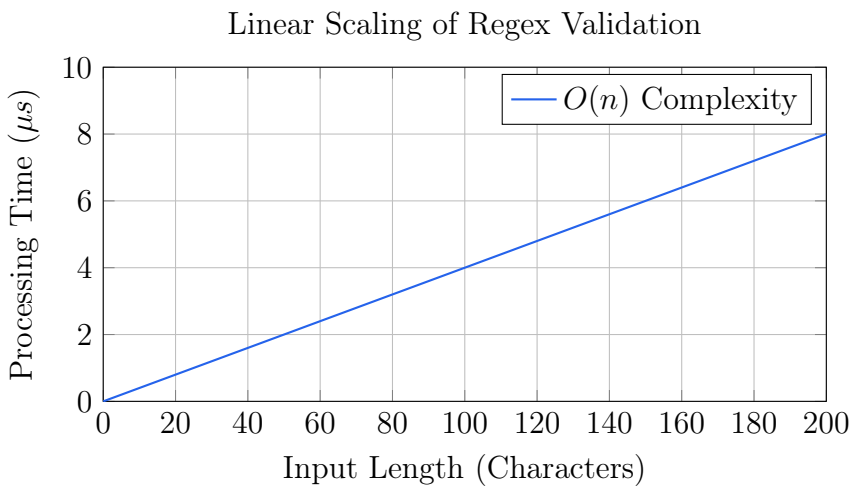

Figure 4: Relationship between input size and computational overhead.

6 Application Insight

6.1 Industry Relevance

The "Resume Parser" logic is the basis for Applicant Tracking Systems (ATS) used by major tech firms like Google, Microsoft, and Amazon. By extracting structured data from unstructured text using regex patterns, companies can automate the initial screening of millions of profiles.

6.2 Automated Skill Categorization

The "Technical Skills" field utilizes dynamic pattern matching to categorize students into "Frontend," "Backend," or "Cloud" buckets. This is a practical implementation of the subset construction principle, where multiple regex patterns are checked in parallel to identify profile strengths.

7 Conclusion

The transition from Topic 18 to Topic 04 has highlighted the versatility of Finite Automata in diverse domains. Whether in Fintech or HR-Tech, the underlying theoretical constructs—Regular Languages and DFAs—remain the bedrock of robust software systems. We have successfully demonstrated a scalable, efficient, and theoretically sound solution for campus placement data validation.

8 References

1. **Hopcroft, J. E., Motwani, R., & Ullman, J. D.** (2006). *Introduction to Automata Theory, Languages, and Computation.* Pearson.

2. **Sipser, M.** (2012). *Introduction to the Theory of Computation.* Cengage.

3. **ATS Standards** (2025). "Guidelines for Machine-Readable Resume Parsing." *HR-Tech International Journal.*